ECEN 5113

HW: 1

Part B

Saira Khan

September 10, 2025

Procedure:

Vivado was used to create, synthesize, simulate, and test each design made in this section. An RTL project is created in Vivado, and design sources and simulation files are added to each project. A behavioral simulation is run to generate a waveform (timing schematic). The default FPGA part used is xc7k70tfbv67-1.

16 to 1 MUX

The first Verilog module represents a 16 to 1 MUX where one of the 16 input signals is selected based on a 4-bit select signal. The module contains an output port (y), a 16-bit data input (i), and a 4-bit select signal (s). An always block with a sensitivity list of "(*)" ensures that the block executes whenever there is a change to any input. Within the always block, a case statement is used to cover all the possible combinations of the select signal, and for each combination, it matches the input to the corresponding output. A testbench is then created to verify the functionality of the 16 to 1 MUX module. The testbench includes signal decorations for the output, input, and select signals. The multiplexer is then instantiated (d) with the declared inputs/outputs. An initial block sets up the inputs and generates the simulation. The testbench is set up with two for loops. The outer one tests four different input patterns, and for each pattern, tests all 16 select values. So when s=k (the bit position tested), y = i[k] (the actual bit value). 10 time units are used as the time between testing each case, giving time for outputs to settle, and it makes it easier to see changes in waveforms.

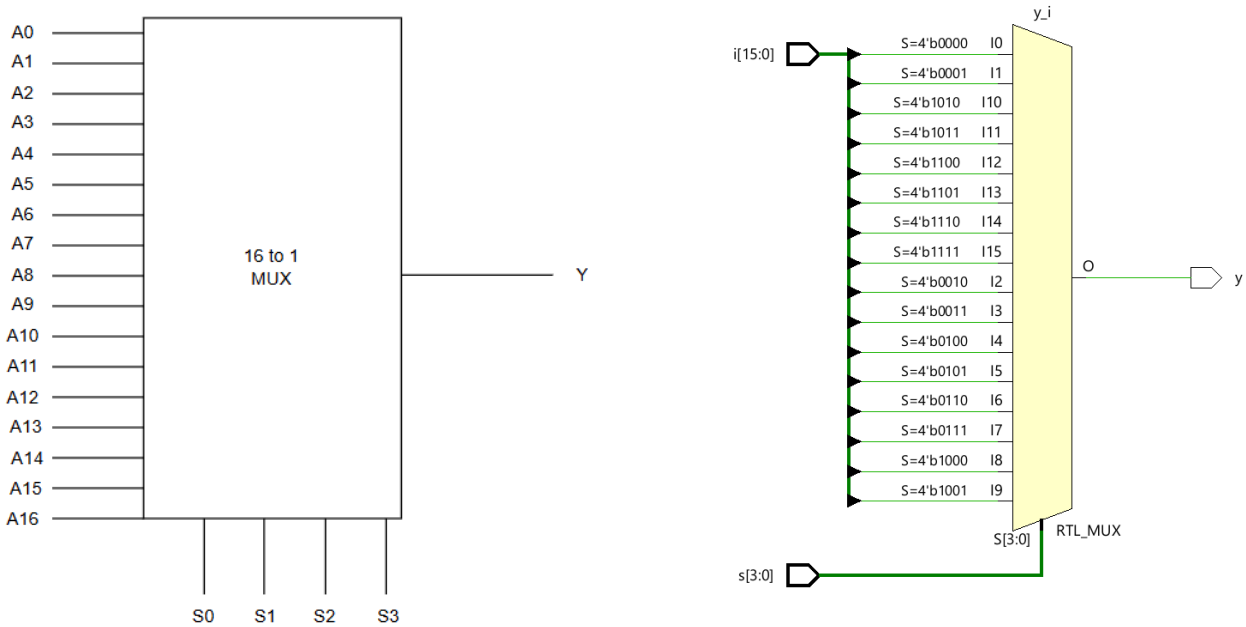

Figure 1: Schematics of a 16 to 1 multiplexer

Table 1: Truth Table of 16 to 1 MUX

S0	S1	S2	S3	Y
0	0	0	0	A0
0	0	0	1	A1
0	0	1	0	A2
0	0	1	1	A3
0	1	0	0	A4
0	1	0	1	A5
0	1	1	0	A6
0	1	1	1	A7
1	0	0	0	A8
1	0	0	1	A9
1	0	1	0	A10
1	0	1	1	A11
1	1	0	0	A12
1	1	0	1	A13
1	1	1	0	A14
1	1	1	1	A15

```verilog
//define module
module mux16(
    //define inputs and outputs
    output reg y, //output - 1 bit default
    input [15:0]i, //16-bit input
    input[3:0]s// 4-bit select

);


//mux behavior -- use case statements
    always @(*) begin //sensitivity list -- active when any input change
        case (s)
            4'b0000: y = i[0];
            4'b0001: y = i[1];
            4'b0010: y = i[2];
            4'b0011: y = i[3];
            4'b0100: y = i[4];
            4'b0101: y = i[5];
            4'b0110: y = i[6];
            4'b0111: y = i[7];
            4'b1000: y = i[8];
            4'b1001: y = i[9];
            4'b1010: y = i[10];
            4'b1011: y = i[11];
            4'b1100: y = i[12];
            4'b1101: y = i[13];
            4'b1110: y = i[14];
            4'b1111: y = i[15];
            default: y = 1'b0;
        endcase
        end
    endmodule
```

```verilog
//testbench

//define module
module mux16_tb;
    //declare input/output
    wire y; //output
    reg [15:0] i; //16 bit input
    reg [3:0] s; // 4 bit select

    //instantiate mux16 module with instance name 'd'
    mux16 d(y, i, s);

    //initial block for setup
    initial
        begin
            //i = 16'b0000000000000000; //initial value for 16 bit input
            //s[0] = 0; // initial value of select bit 0
            //s[1] = 0; //initial value of select bit 1
            //s[2] = 0; //initial value of select bit 2
            //s[3] = 0; //initial value of select bit 3

            //try 4 different input patterns to test mux behavior - outer loop
            for (integer j = 0; j <4; j = j+1)
                begin
                    case(j)
                        0: i = 16'h0000; //all zeros
                        1: i = 16'hFFFF; // all ones
                        2: i = 16'hA5A5; //alternating ones and zeros
                        3: i = 16'h3214; //random
                    endcase

                //go through all select values - s 0-15
                for (integer k = 0; k <16; k = k+1)
                    begin
                        s = k; //set select to current test value k
                        #10; //wait 10 time units
                        //display test results
                        $display("time=%0t | s=%0d | y=%b | i[%0d]=%b",
                          $time, s, y, k, i[k]);
                    end
                end
            //end simulation after all tests
            $finish;
        end

    endmodule
```

Figure 2: Verilog code and testbench for 16 to 1 MUX

3 to 8 Decoder

The second logic module is a 3 to 8 decoder, which has 3 inputs and 8 (2^3) output lines. When the decoder is enabled, one of the eight outputs will be active for each combination of inputs. In the module, an input wire for enable (en) is defined as well as a 3-bit input (s) and an 8-bit output register (y). The output is defined as a register due to the nature of always blocks, because the always block needs to store the value between sensitivity list changes. An always block has a sensitivity list of "(en, s)" which activates when the enable or input is changed. Inside the always block, if the enable is high, the decoder is active, and a case statement is used to match all possible inputs with their corresponding outputs. A statement is made when the enable is low (0), allowing the output to be 0. A testbench is created to test and verify that the decoder module's logic functions correctly. The enable and input signals are made as registers because they need to store different values as they change, and the output is declared as a wire. The module is instantiated as "d", and an initial block sets up the main logic for the testbench. The decoder is set up to be enabled, and a for loop goes through all the possible inputs. Another case is created to ensure that the enable is tested when it is low, and the input is changed to ensure that it is not just being read as the "0" it was set to earlier.

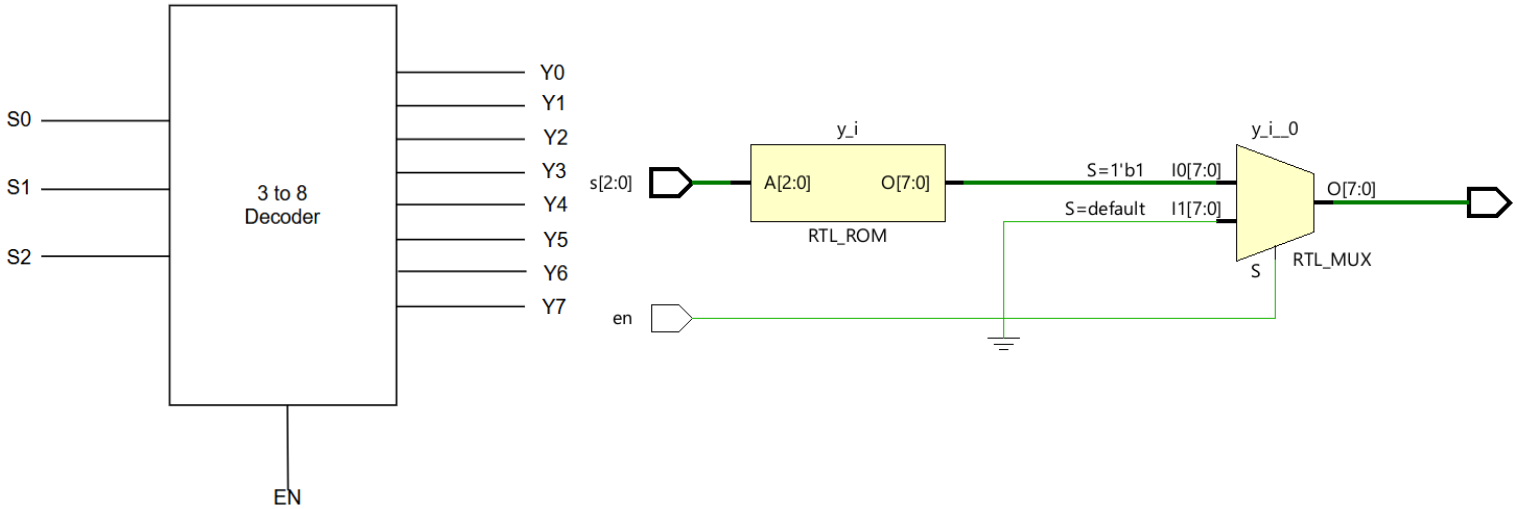

Figure 3: Schematics of a 3 to 8 Decoder

Table 2: Truth Table of a 3 to 8 Decoder

S0	S1	S2	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

```
//define module
module decoder38(en, s, y);
    //input port
    input en; //1 bit enable -- defaults to wire
    input [2:0] s; //3 bit input

    //output - must be reg bc of always block
    output reg [7:0] y;//8 bit output

    //always block to trigger activation
    //active when en or one of 3 inputs change
    always @(en or s)
        begin
            if(en==1) //if enable high, decoder = active
                begin
                    case(s)
                    //truth table
                        3'b000: y = 8'b00000001;
                        3'b001: y = 8'b00000010;
                        3'b010: y = 8'b00000100;
                        3'b011: y = 8'b00001000;
                        3'b100: y = 8'b00010000;
                        3'b101: y = 8'b00100000;
                        3'b110: y = 8'b01000000;
                        3'b111: y = 8'b10000000;
                        default: y = 8'b00000000; //default case
                    endcase
                end
            //if enable is low, outputs = low
            else
                begin
                    y = 8'b00000000;
                end
            end
    endmodule
```

```
//testmodule

//define module
module decoder38_tb;
    //declare input/output
    reg en; //enable
    reg [2:0] s; //input
    wire [7:0] y; //output

    //instantiate module
    decoder38 d (en, s, y);

    //initial block for setup
    initial
        begin
            //initalize inputs
            en = 1'b1; //enable the decoder
            s = 3'b000; // initialize input

            //input combinations

            for (integer i = 0; i <8; i = i + 1)
                begin
                    s = i; //current input value
                    #10; //wait 10 time units for a stable output
                    $display("time=%0d  | en=%b | s=%3b  |y=%9b",
                            $time, en, s, y);
                end

            //when enable is low
            en = 1'b0;
            s= 3'b101; //change input
            #10;
            $display("time=%0d  |en=%b |  s=%3b  |y=%8b",
                            $time, en, s, y);

            $finish; //end simulation
        end
    endmodule
```

Figure 4: Verilog Code and testbench for 3 to 8 Decoder

1-bit Full Adder

A 1-bit full adder performs the addition of three 1-bit binary numbers (two inputs: A, B, and a carry-in bit: Cin). It produces the two outputs: Sum and Carry-out. The sum output of a full adder can be reduced to the logical expression:

$$S = A \oplus B \oplus Cin$$

The logical expression for Cout can be written as:

$$Cout = AB + Cin(A \oplus B)$$

The module is defined with the inputs a, b, and cin, and the outputs sum and cout. An always block is created with the sensitivity list "(a, b, cin)" so that the block activates when any input changes value. The sum and cout are calculated using the logical expressions above in Verilog language. A testbench is made to simulate the behavior of the full adder by defining the inputs and outputs, instantiating the module, and using an initial block for the main logic. Inside the initial block, the input values (a, b, and cin) are changed, reflecting the truth table for a full adder. These values, along with the values of sum and cout, are then displayed.

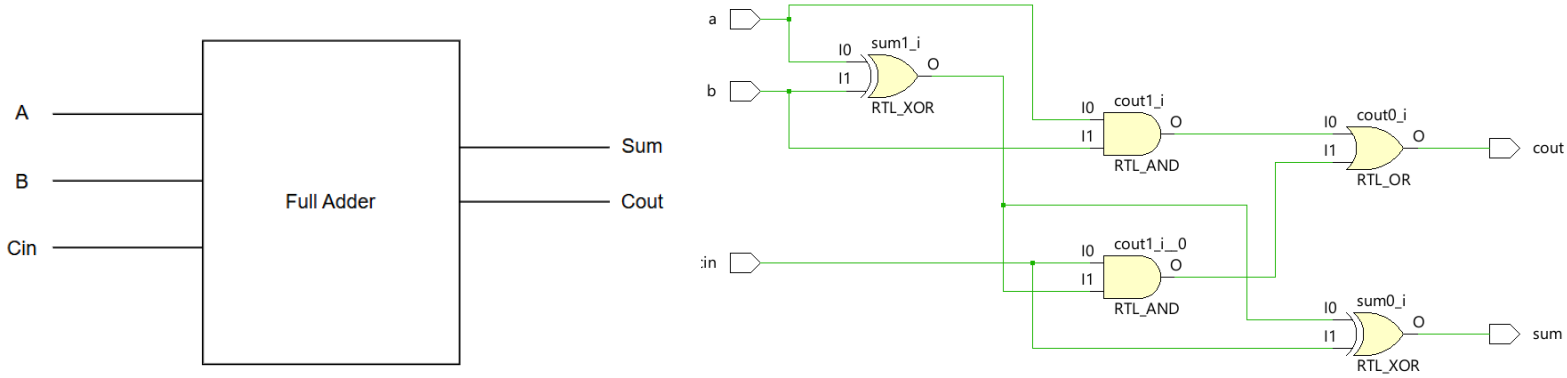

Figure 5: Schematics of a Full Adder

Table 3: Truth Table of a Full Adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

```verilog
//define module
module fulladder (
    //define inputs/outputs
    input a,
    input b,
    input cin,
    output reg sum,
    output reg cout
);

    //always block for changes in input
    always @(a, b, cin)
        begin
            //calculate the sum
            //use xor (^)
            sum = a ^ b ^ cin;

            //calculate the carry out
            cout = (a & b) | (cin & (a ^ b));
        end
endmodule
```

```verilog
//testmoudle

//define module
module fulladder_tb;
    //declare input/output
    reg a;
    reg b;
    reg cin;
    wire sum;
    wire cout;

    //instantiate module
    fulladder d(a, b, cin, sum, cout);

    //initial block for setup
    initial
        begin
            //initalize inputs
            a = 0;
            b = 0;
            cin = 0;

            //test possible input combinations - truth table
            //wait 10 unit times between & display variables
            #10 a = 0; b=0; cin=0;
            $display("%b %b %b | %b  %b",
            a, b, cin, sum, cout);
            #10 a = 0; b=0; cin=1;
            $display("%b %b %b | %b  %b",
            a, b, cin, sum, cout);
            #10 a = 0; b=1; cin=0;
            $display("%b %b %b | %b  %b",
            a, b, cin, sum, cout);
            #10 a = 0; b=1; cin=1;
            $display("%b %b %b | %b  %b",
            a, b, cin, sum, cout);
            #10 a = 1; b=0; cin=0;
            $display("%b %b %b | %b  %b",
            a, b, cin, sum, cout);
            #10 a = 1; b=0; cin=1;
            $display("%b %b %b | %b  %b",
            a, b, cin, sum, cout);
            #10 a = 1; b=1; cin=0;
            $display("%b %b %b | %b  %b",
            a, b, cin, sum, cout);
            #10 a = 1; b=1; cin=1;
            $display("%b %b %b | %b  %b",
            a, b, cin, sum, cout);


            $finish; //finish simulation
        end
endmodule
```

Figure 6: Verilog code and testbench for Full Adder

<u>*D Flip Flop*</u>

A D flip-flop is a clocked sequential logic circuit that stores one bit of data. It delays the data input (D) to become the output (Q) when the clock signal transitions to an active edge. The D input is held at the output until the next active clock edge. This module defines a D flip-flop with an asynchronous reset, which means that when the reset is active, the output will immediately be set to 0 without waiting for the clock. The inputs d, clk, and rst are defined as well as the outputs q and q'. An always block is activated with the positive edge of the clock or reset. If reset is active, then the output immediately sets to 0 (q = 0 and q' = 1). Otherwise, if the reset is not active, then the output q takes the value of d (input) on the positive edge of the clock (and q' is the complement of q). A testbench is made to verify this logic. Inputs and outputs are declared, and the module is instantiated. An initial block is made to set up the clock. The clock is set to 0 at time 0. A forever loop is declared (infinite loop), and the clock signal is toggled every 10 time units. So positive edges (when the d flip-flop logic should be activated) are at times 10, 30, 50, 70, 90, etc., while negative edges are at 20, 40, 60, 80, 100, etc.. Another initial block is created to simulate the logic of the D flip-flop. The inputs are initialized so that the input is equal to 0 and the reset is active, so q should be 0. The reset is then de-asserted to test the rest of the logic. Table 4 shows the expected output of Q and Q' based on the clock, D input, reset, and the testbench created for this Verilog module.

Table 4: Expected Timeline of D Flip-Flop (based on testbench)

Time	Clk	D	Rst	Q	Q’	Event
0	0	0	1	0	1	Start in reset
5	0	0	0	0	1	Reset deasserted
10	1	0	0	0	1	Posedge: Q = 0
20	0	1	0	0	1	D set to 1
25	0	1	0	0	1	Waiting for posedge
30	1	1	0	1	0	Q = 1
40	1	0	0	1	0	D =0
50	1	0	0	0	1	Q=0
60	0	1	0	0	1	D = 1
70	1	1	0	1	0	Q=1
70	1	1	1	0	1	Reset asserted Q=0
72	1	1	0	0	1	Reset deasserted
90	1	1	0	1	0	Q=1

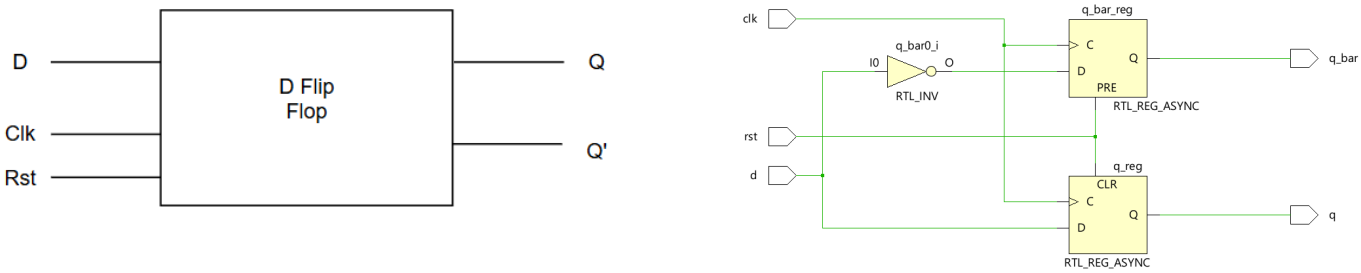

Figure 7: Schematics of a D flip-flop

Table 5: Truth Table of a D flip-flop

Rst	D	Clk	Q	Q’
1	x	x	0	1
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0

```
//define module
module dflipflop(
    //define inputs/outputs
    input clk, //clock input
    input d, //data input
    input rst, //reset input asynch
    output reg q, //output
    output reg q_bar //complement of q
);

    //always block to model logic
    //triggered on positive edge of clock or posedge of reset
    always @(posedge clk or posedge rst)
        begin
        //if reset - q becomes 0 immediatly
        //rst overrides everything else
        if (rst)
            begin
                q <= 0;
                q_bar <=1;
            end

            //normal operation
        else
            begin
                // q takes value of d on positive edge of clk
                q <= d;
                q_bar <= ~d;
            end
        end
endmodule
```

Figure 8: Verilog code for D flip-flop

```
//testmodule

//define module
module dflipflop_tb;

    //define inputs/outputs
    reg clk;
    reg d;
    reg rst;
    wire q;
    wire q_bar;

    //instantitate module
    dflipflop s(clk, d, rst, q, q_bar);

    //initial block to setup clock
    initial
        begin
            //initalize input
            clk = 0;
            //toggles clock every 10 time units
            forever #10 clk = ~clk;
        end

    //simulate initial block
    initial
        begin
            //initalize inputs
            d = 0;
            rst = 1; //start in reset
            #5; //wait some time

            //de-assert rst
            rst = 0;
            #15; //wait a bit

            //input changes

            d = 1; //set d = 1 ; q still 0 bc no clock edge
            #5;
            $display("time=%0t  d=%b clk=%b | q=%b  q_bar=%b",
            $time,d, clk, q, q_bar);
            #15;
            $display("time=%0t  d=%b clk=%b | q=%b  q_bar=%b",
            $time,d, clk, q, q_bar);


            d=0; //set d = 0 q = 1
            #10;
            $display("time=%0t  d=%b clk=%b | q=%b  q_bar=%b",
            $time,d, clk, q, q_bar);
            #10;
            $display("time=%0t  d=%b clk=%b | q=%b  q_bar=%b",
            $time,d, clk, q, q_bar);

            d=1; //set d = 1 ; q = 0
            #10;
            $display("time=%0t  d=%b clk=%b | q=%b  q_bar=%b",
            $time,d, clk, q, q_bar);


            //test reset while q=1
            rst = 1; //q should immediatly become 0
            #2;
            $display("time=%0t  d=%b clk=%b | q=%b  q_bar=%b",
            $time,d, clk, q, q_bar);

            rst = 0; //deassert reset
            #18;
            $display("time=%0t  d=%b clk=%b | q=%b  q_bar=%b",
            $time,d, clk, q, q_bar);



            $finish; //finish simulation
        end
endmodule
```

Figure 9: Testbench code for a D flip-flop

<u>Observations:</u>

Simulating the logic modules in Vivado brought up a common issue: timing. I realized that if there were no delays between a change in input and the display of the input, sometimes the output displayed would be wrong. This fault was not due to a logic error or coding problem, but a timing issue. With the full adder module, not having at least 1 time unit pause between changing the inputs and displaying the outputs meant that the output was wrong. Table 6 shows the output I got when there were no time delays between changing an input and displaying the output. From the table, it can be seen that past outputs would be displayed even if the input was changed. For example, A = 0, B = 1, Cin = 0, the output is correct. But, A = 0, B=1, Cin = 1, the output is wrong. In fact, the output is the same as the last output.

Table 6: Wrong Output Received from Full Adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	0	1
1	1	1	0	1

```
#10 a = 0; b=0; cin=0;
#1;
$display("%b %b %b | %b  %b",
a, b, cin, sum, cout);
#10 a = 0; b=0; cin=1;
#1;
$display("%b %b %b | %b  %b",
a, b, cin, sum, cout);
#10 a = 0; b=1; cin=0;
#1;
$display("%b %b %b | %b  %b",
a, b, cin, sum, cout);
#10 a = 0; b=1; cin=1;
#1;
$display("%b %b %b | %b  %b",
a, b, cin, sum, cout);
#10 a = 1; b=0; cin=0;
#1;
$display("%b %b %b | %b  %b",
a, b, cin, sum, cout);
#10 a = 1; b=0; cin=1;
#1;
$display("%b %b %b | %b  %b",
a, b, cin, sum, cout);
#10 a = 1; b=1; cin=0;
#1;
$display("%b %b %b | %b  %b",
a, b, cin, sum, cout);
#10 a = 1; b=1; cin=1;
#1;
$display("%b %b %b | %b  %b",
a, b, cin, sum, cout);
```

Figure 10: How delay was added to Full Adder testbench

Timing was critical for the D flip-flop logic module as well. I had to change the delays between the input changes and displays a couple of times before I got the timing to simulate a D flip-flop properly. Without the correct timing, I was getting a similar problem to that of the Full Adder. Though fixing this timing was more complicated due to the fact that a clock is used in this logic module, and the output depends on it. This D flip-flop's output only changes on the positive edge of the clock, so this needed to be reflected in the testbench code. Figure 19 shows how the output, Q, did not change to the desired value right away (at time 90000). The positive edge

should occur at t=90, and reset is deasserted, and D equals 1, Q should be equal to 1 at that time. Figure 19 shows that it is equal to 0. However, Figure 11 shows the correct output 2 time units later, which suggests a timing issue.

```
time=25000  d=1 clk=0 | q=0  q_bar=1
time=40000  d=1 clk=1 | q=1  q_bar=0
time=50000  d=0 clk=0 | q=1  q_bar=0
time=60000  d=0 clk=1 | q=0  q_bar=1
time=70000  d=1 clk=0 | q=0  q_bar=1
time=72000  d=1 clk=1 | q=0  q_bar=1
time=92000  d=1 clk=1 | q=1  q_bar=0
```

Figure 11: Right output of D flip-flop

<u>Results:</u>

<u>16 to 1 MUX:</u>

Figure 12 shows the waveform generated by the 16 to 1 MUX code. It shows 4 distinct time periods that correspond to the four test patterns (each lasts 160ns). From 0-160ns, the input is all zeroes (black). It changes to all ones (green) from 160-320ns. An alternating pattern can be seen from 320-480ns (mix of black and green). Lastly, from 480-640ns a random pattern can be seen. At the bottom of the waveform is the select signal, counting from 0 to 15 each test period. The output, y, can be verified by looking at the waveform. In the first period, with the zeroes, the output stays low. While in the second period, the output stays high because the input values are all ones. Figure 13 shows the output log from Vivado when the logic module is simulated. The logic can be verified by looking at this output as well.

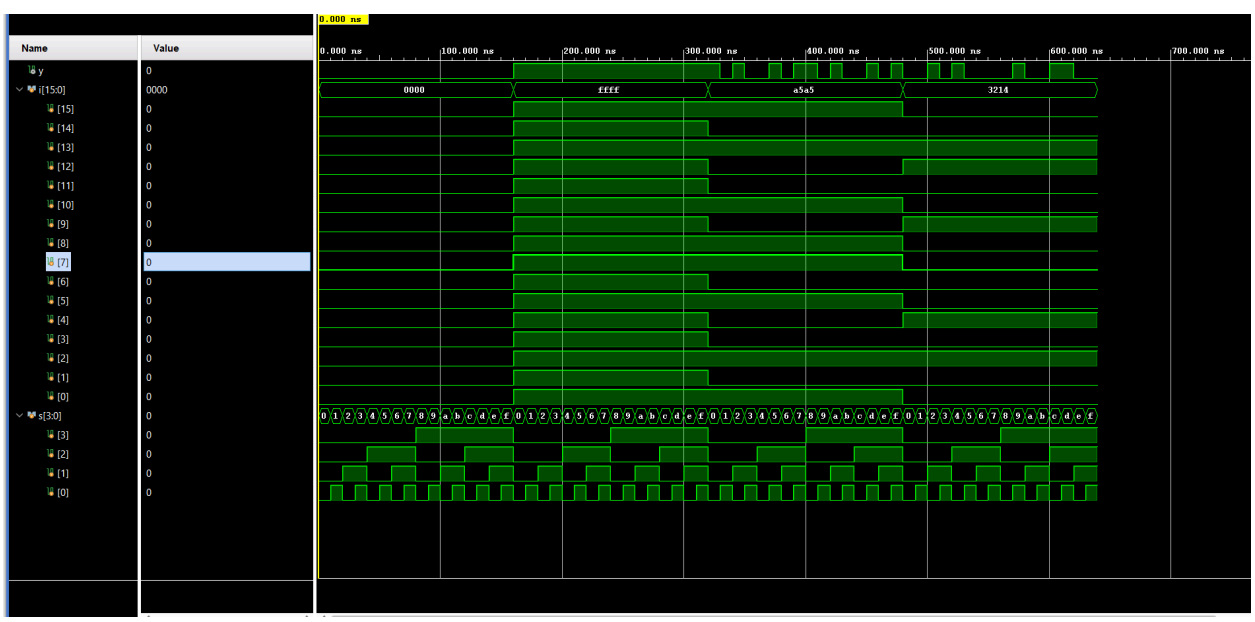

Figure 12: Waveform of 16 to 1 MUX

```
time=10000 | s=0 | y=0 | i[0]=0
time=20000 | s=1 | y=0 | i[1]=0
time=30000 | s=2 | y=0 | i[2]=0
time=40000 | s=3 | y=0 | i[3]=0
time=50000 | s=4 | y=0 | i[4]=0
time=60000 | s=5 | y=0 | i[5]=0
time=70000 | s=6 | y=0 | i[6]=0
time=80000 | s=7 | y=0 | i[7]=0
time=90000 | s=8 | y=0 | i[8]=0
time=100000 | s=9 | y=0 | i[9]=0
time=110000 | s=10 | y=0 | i[10]=0
time=120000 | s=11 | y=0 | i[11]=0
time=130000 | s=12 | y=0 | i[12]=0
time=140000 | s=13 | y=0 | i[13]=0
time=150000 | s=14 | y=0 | i[14]=0
time=160000 | s=15 | y=0 | i[15]=0
time=170000 | s=0 | y=1 | i[0]=1
time=180000 | s=1 | y=1 | i[1]=1
time=190000 | s=2 | y=1 | i[2]=1
time=200000 | s=3 | y=1 | i[3]=1
time=210000 | s=4 | y=1 | i[4]=1
time=220000 | s=5 | y=1 | i[5]=1
time=230000 | s=6 | y=1 | i[6]=1
time=240000 | s=7 | y=1 | i[7]=1
time=250000 | s=8 | y=1 | i[8]=1
time=260000 | s=9 | y=1 | i[9]=1
time=270000 | s=10 | y=1 | i[10]=1
time=280000 | s=11 | y=1 | i[11]=1
time=290000 | s=12 | y=1 | i[12]=1
time=300000 | s=13 | y=1 | i[13]=1
time=310000 | s=14 | y=1 | i[14]=1
time=320000 | s=15 | y=1 | i[15]=1
time=330000 | s=0 | y=1 | i[0]=1
```

```
time=330000 | s=0 | y=1 | i[0]=1
time=340000 | s=1 | y=0 | i[1]=0
time=350000 | s=2 | y=1 | i[2]=1
time=360000 | s=3 | y=0 | i[3]=0
time=370000 | s=4 | y=0 | i[4]=0
time=380000 | s=5 | y=1 | i[5]=1
time=390000 | s=6 | y=0 | i[6]=0
time=400000 | s=7 | y=1 | i[7]=1
time=410000 | s=8 | y=1 | i[8]=1
time=420000 | s=9 | y=0 | i[9]=0
time=430000 | s=10 | y=1 | i[10]=1
time=440000 | s=11 | y=0 | i[11]=0
time=450000 | s=12 | y=0 | i[12]=0
time=460000 | s=13 | y=1 | i[13]=1
time=470000 | s=14 | y=0 | i[14]=0
time=480000 | s=15 | y=1 | i[15]=1
time=490000 | s=0 | y=0 | i[0]=0
time=500000 | s=1 | y=0 | i[1]=0
time=510000 | s=2 | y=1 | i[2]=1
time=520000 | s=3 | y=0 | i[3]=0
time=530000 | s=4 | y=1 | i[4]=1
time=540000 | s=5 | y=0 | i[5]=0
time=550000 | s=6 | y=0 | i[6]=0
time=560000 | s=7 | y=0 | i[7]=0
time=570000 | s=8 | y=0 | i[8]=0
time=580000 | s=9 | y=1 | i[9]=1
time=590000 | s=10 | y=0 | i[10]=0
time=600000 | s=11 | y=0 | i[11]=0
time=610000 | s=12 | y=1 | i[12]=1
time=620000 | s=13 | y=1 | i[13]=1
time=630000 | s=14 | y=0 | i[14]=0
time=640000 | s=15 | y=0 | i[15]=0
```

Figure 13: Verilog Output of 16 to 1 MUX

3 to 8 Decoder

Figure 15 shows the output from Vivado of the simulated logic module. The output shows the intended behavior. When the enable is active and the inputs change, the output changes to the corresponding number. When the enable is not active, the output goes to 0 regardless of what the input is. From the waveform, the select inputs can be seen cycling through different binary values. The green highlighting shows the active signals. The output of the decoder can be seen clearly here, with its step-like shape that reflects the truth table.

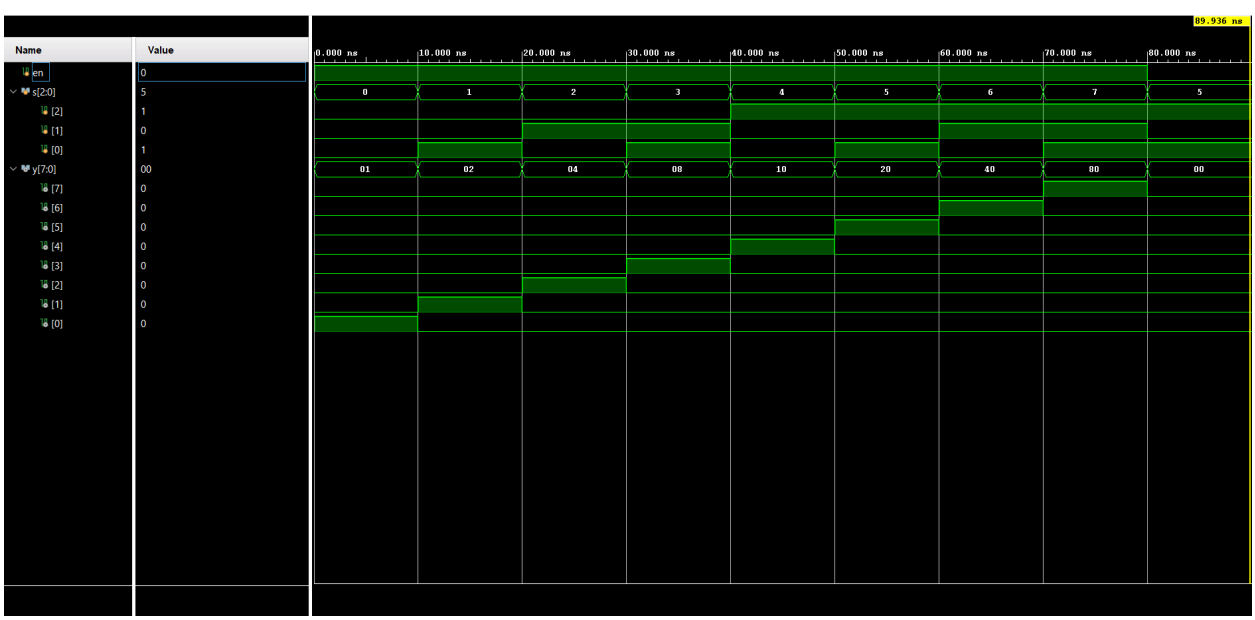

Figure 14: Waveform of 3 to 8 Decoder

```
# }
# run 1000ns
time=10  | en=1 | s=000  |y=000000001
time=20  | en=1 | s=001  |y=000000010
time=30  | en=1 | s=010  |y=000000100
time=40  | en=1 | s=011  |y=000001000
time=50  | en=1 | s=100  |y=000010000
time=60  | en=1 | s=101  |y=000100000
time=70  | en=1 | s=110  |y=001000000
time=80  | en=1 | s=111  |y=010000000
time=90  |en=0 |  s=101  |y=00000000
$finish called at time : 90 ns : File
```

Figure 15: Verilog Output of 3 to 8 Decoder

1-bit Full Adder

Figure 17 shows the output in Vivado of the Full Adder logic module. It shows the intended behavior and is formatted like so: A, B, cin, Sum, cout. The waveform in Figure 16 shows the correct behavior as well. It can be seen from the waveform that whenever there is an odd number of inputs equal to 1, the sum goes high (1).

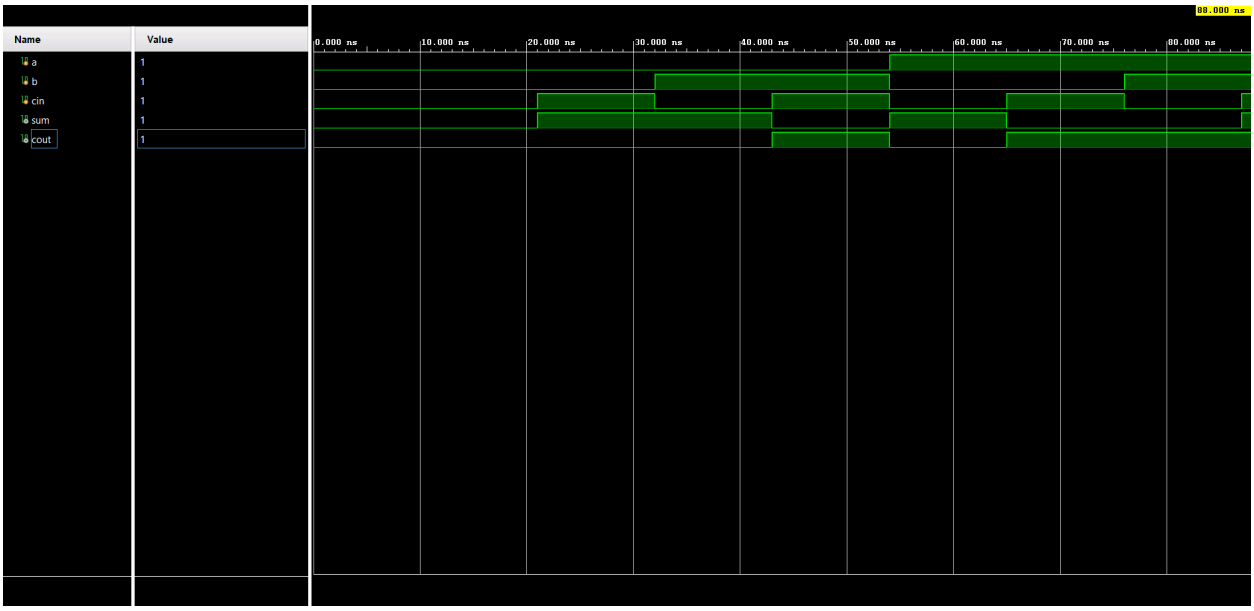

Figure 16: Waveform of 1-bit Full Adder

```
# run 1000ns
0 0 0 | 0  0
0 0 1 | 1  0
0 1 0 | 1  0
0 1 1 | 0  1
1 0 0 | 1  0
1 0 1 | 0  1
1 1 0 | 0  1
1 1 1 | 1  1
$finish called at time : 88 ns
```

Figure 17: Output of 1-bit Full Adder

D Flip-flop

Figure 19 shows the output of the simulated logic module. The behavior is correct except for the last line, which is talked about in the Observation section. Looking at Figure 18, the reset is initially high, which is the intended behavior of the testbench that was written. It can also be seen that whenever the clock input goes high, the value of Q changes depending on D. For example, at 30ns, the clock goes high and D is already 1, so Q changes to the value one and remains there until the next positive clock edge at 50ns. Looking at how Q changes with the positive clock edge confirms that the behavior of this module is correct.

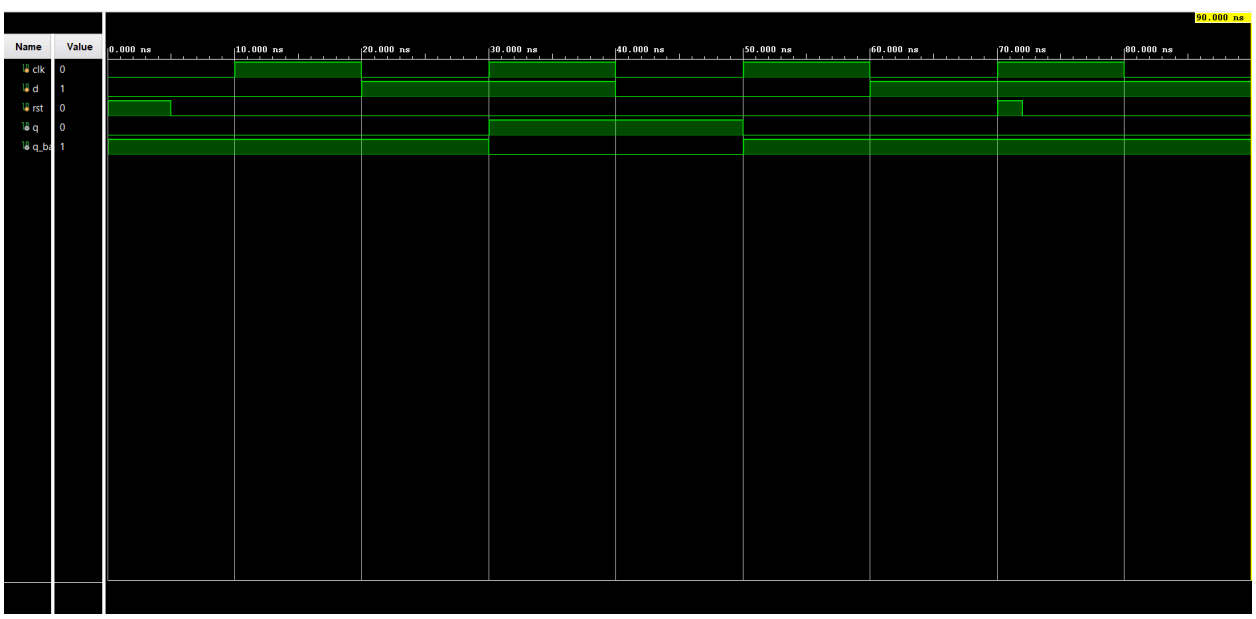

Figure 18:Waveform of D flip-flop

```
# run 1000ns
time=25000  d=1 clk=0 | q=0  q_bar=1
time=40000  d=1 clk=1 | q=1  q_bar=0
time=50000  d=0 clk=0 | q=1  q_bar=0
time=60000  d=0 clk=1 | q=0  q_bar=1
time=70000  d=1 clk=0 | q=0  q_bar=1
time=72000  d=1 clk=1 | q=0  q_bar=1
time=90000  d=1 clk=0 | q=0  q_bar=1
```

Figure 19: Output of D flip-flop

Conclusion:

Part B goes through simple logic modules modeled using Verilog. Behavioral Verilog is mostly used to write the modules in this section, showing the logic that goes behind each module. From creating and simulating these modules, the idea of timing and adding delays to testbenches was shown to be extremely important. Without proper timing, the behavior of the module can be incorrect regardless of the actual code/logic.